GRENOBLE-INP ENSIMAG

INGÉNIERIE DES SYSTÈMES D'INFORMATION

2A ISI

GRENOBLE-INP ENSIMAG

INFORMATION SYSTEMS ENGINEERING

2A ISI



ÉCOLE NATIONALE SUPÉRIEURE D'INFORMATIQUE ET DE MATHÉMATIQUES APPLIQUÉES DE GRENOBLE

PROJET GoMile

Rapport

Réalisé par :

DUONG DANG KHOA DANG
DIAKITE ALPHA OUSMANE
JOUINI YOUSSEF
MAOUDE SAMIR
OUDRHIRI IDRISSI SAFWANE

CLIENT :

GoMile Entreprise

2026–2027

Table des matières

1 Introduction **3**

2 Manuel utilisateur **5**

1 App mobile 5

1.1 But de l'application 5

1.2 Installation et première ouverture 5

1.3 Inscription (par étapes) 7

1.4 Écran principal — Tableau de bord 7

1.5 Liste des missions 9

1.6 Focus mission et navigation 11

1.7 Détail d'une mission 13

1.8 Portefeuille (Wallet) 15

1.9 Profil et vérification KYC 15

3 Documentation de conception **17**

1 Backend et API 17

2 Interface Web 17

2.1 Partie Administrateur 17

2.2 Partie Commerciale : 17

3 Application Mobile Livreur 18

3.1 Rôle de la partie mobile 18

3.2 Architecture générale et technologies 18

3.3 Parcours d'inscription 19

3.4 Connexion et vérification email 21

3.5 Réinitialisation du mot de passe 22

3.6 Diagramme d'état global authentification 23

3.7 Décisions UX abandonnées ou déplacées 23

3.8 Navigation et écrans post-connexion 24

3.9 Écran enfant : Détails Mission (MissionDetailsScreen) 26

3.10 Diagramme de flux : Navigation Tableau de Bord → Détails Mission → Focus . . . 28
3.11 Diagramme d'état : Cycle de vie d'une mission acceptée . . . 29
3.12 Écran 2 : Missions (MissionsScreen) . . . 30
3.13 Écran enfant : Focus Mission (MissionFocusScreen) . . . 31
3.14 Diagramme de flux : Acceptation mission → Focus → Vérification codes . . . 34
3.15 Diagramme d'état : Transition états mission dans MissionFocusScreen 35
3.16 Écran 3 : Portefeuille (WalletScreen) . . . 36
3.17 Diagramme de flux : Wallet (solde → retrait → confirmation) . . . 37
3.18 Écran 4 : Profil (ProfileScreen) . . . 37
3.19 Diagramme d'état : Profil et transitions KYC . . . 39
3.20 Écran enfant : KYC (KYCScreen) . . . 40
3.21 Services et stores partagés . . . 45
4 Tests de l'application mobile . . . 48
4.1 Ce qui est testé . . . 48
4.2 Ce que montrent les résultats . . . 49
4.3 Lecture rapide du dossier de tests . . . 49
4.4 Couverture des tests (coverage) . . . 49

Chapitre 1

Introduction

Contexte du projet

GoMile est une plateforme de livraison développée dans le cadre d'un projet de deuxième année à l'École Nationale Supérieure d'Informatique et de Mathématiques Appliquées de Grenoble (ENSIMAG), filière Ingénierie des Systèmes d'Information. Ce projet a été réalisé pour le client *GoMile Entreprise* par une équipe de cinq étudiants : Duong Dang Khoa Dang, Alpha Ousmane Diakite, Youssef Jouini, Samir Maoude et Safwane Oudrhiri Idrissi.

L'objectif de GoMile est de mettre en relation des commerçants ayant besoin de livrer des colis avec des livreurs indépendants disponibles à proximité. La plateforme coordonne l'ensemble du cycle de livraison : création de mission côté commerçant, attribution et exécution par le livreur, validation par code, et reversement des gains sur un portefeuille numérique.

Architecture du système

Le système GoMile repose sur plusieurs composants complémentaires :

— **Application mobile livreur** (React Native / Expo) : permet au livreur de s'inscrire, de passer une vérification d'identité (KYC), de consulter et d'accepter des missions, de naviguer jusqu'au point de livraison et de gérer ses gains depuis un portefeuille intégré.

— **Interface web commerciale** : espace destiné aux commerçants pour créer et suivre leurs missions de livraison.

— **Interface web d'administration** : tableau de bord permettant à l'équipe GoMile de superviser l'ensemble de la plateforme, de valider les comptes livreurs et de gérer les litiges.

— **Backend et API REST** : cœur métier du système, exposant l'ensemble des fonctionnalités aux différents clients via une API unifiée.

Structure du rapport

Ce document constitue le rapport technique complet du projet. Il est organisé en quatre grandes parties.

Le **manuel utilisateur** présente les fonctionnalités de l'application mobile du point de vue du livreur : installation, inscription en plusieurs étapes, tableau de bord, gestion des missions, portefeuille et profil KYC.

La **documentation de conception** détaille les choix d'architecture et d'implémentation pour chaque composant : le backend et son API, les interfaces web administrateur et commerciale, ainsi que l'application mobile et ses briques techniques (Zustand, React Navigation, services réseau).

La section **tests et validation** rend compte de la stratégie de test adoptée pour l'application mobile : 276 tests répartis en 24 suites, couvrant les stores Zustand, les clients réseau, les hooks métier et les composants réutilisables, avec un taux de couverture supérieur à 93 % sur les couches critiques.

Enfin, la section **plugin** documente les éventuels greffons ou extensions développés en complément de la plateforme principale.

Chapitre 2

Manuel utilisateur

1 App mobile

1.1 But de l'application

Cette application mobile s'adresse aux livreurs : elle permet de s'inscrire, de se connecter, de recevoir et d'accepter des missions, de suivre un trajet, de gérer son portefeuille et son profil.

1.2 Installation et première ouverture

Procédure rapide depuis la racine du projet (développeur/testeur) :

1. Installer `pnpm` si nécessaire :
   — Option simple : `npm install -g pnpm`
   — Ou activer Corepack et préparer pnpm : `corepack enable && corepack prepare pnpm@latest -activate`
2. Installer toutes les dépendances (depuis la racine du dépôt) : `pnpm install`.
3. Lancer en mode Tunnel pour tester sur un vrai téléphone via l'App Expo Go :
   — `pnpm run mobile:tunnel` (ou `corepack pnpm run mobile:tunnel`)
4. OU Lancer l'application mobile en mode LAN (réseau local) si vous voulez utiliser un simulateur sur PC :
   — `pnpm run mobile:lan` (ou `corepack pnpm run mobile:lan`)
5. Installer Expo Go sur le téléphone OU configurer un émulateur sur le PC. Expo Go est recommandé pour les tests réels (plus simple et fiable).

Après ces étapes, ouvrir l'application via Expo (QR code ou lien) ; l'utilisateur arrive sur l'écran de connexion.



FIGURE 2.1 – Écran de connexion — saisie email / mot de passe ou démarrer l'inscription

1.3 Inscription (par étapes)

L'inscription se déroule en plusieurs étapes : informations de base, identité (photo, nom, prénom, date de naissance, téléphone) et logistique (adresse, véhicule). Les captures ci-dessous montrent les étapes.

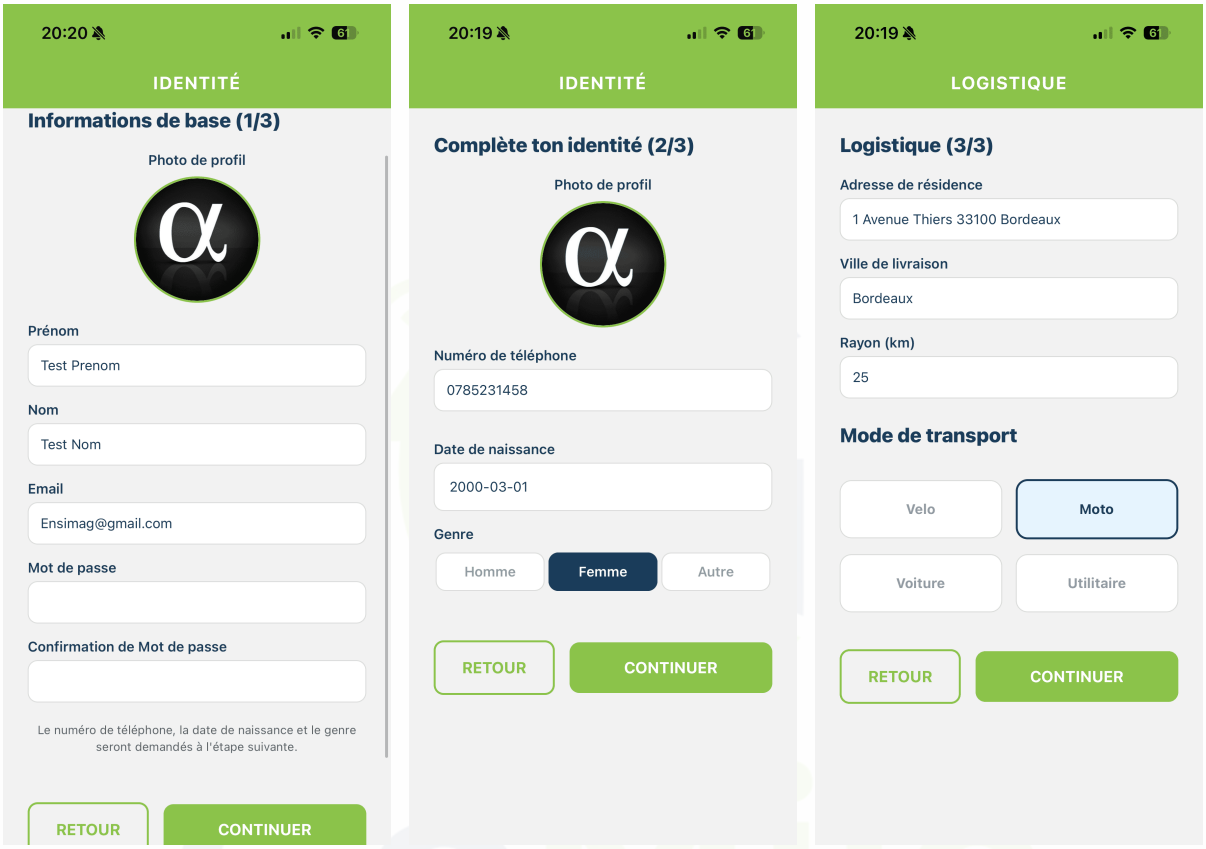


FIGURE 2.2 – Étapes d'inscription 1/3, 2/3, 3/3

1.4 Écran principal — Tableau de bord

Une fois connecté et validé, l'application ouvre le tableau de bord. Il affiche la carte, le solde rapide du jour et les missions proposées à proximité.



FIGURE 2.3 – Tableau de bord — carte et cartes de missions proposées

1.5 Liste des missions

L'onglet "Missions" regroupe : notifications, missions en cours et missions à proximité. Chaque carte propose un aperçu rapide et un bouton *Voir les détails*.



FIGURE 2.4 – Écran Missions — liste et état

1.6 Focus mission et navigation

Lorsqu'une mission est sélectionnée, l'écran Mission Focus montre le tracé sur la carte, les points pickup / delivery et les actions principales (valider identification commerçant, valider code client, terminer mission).

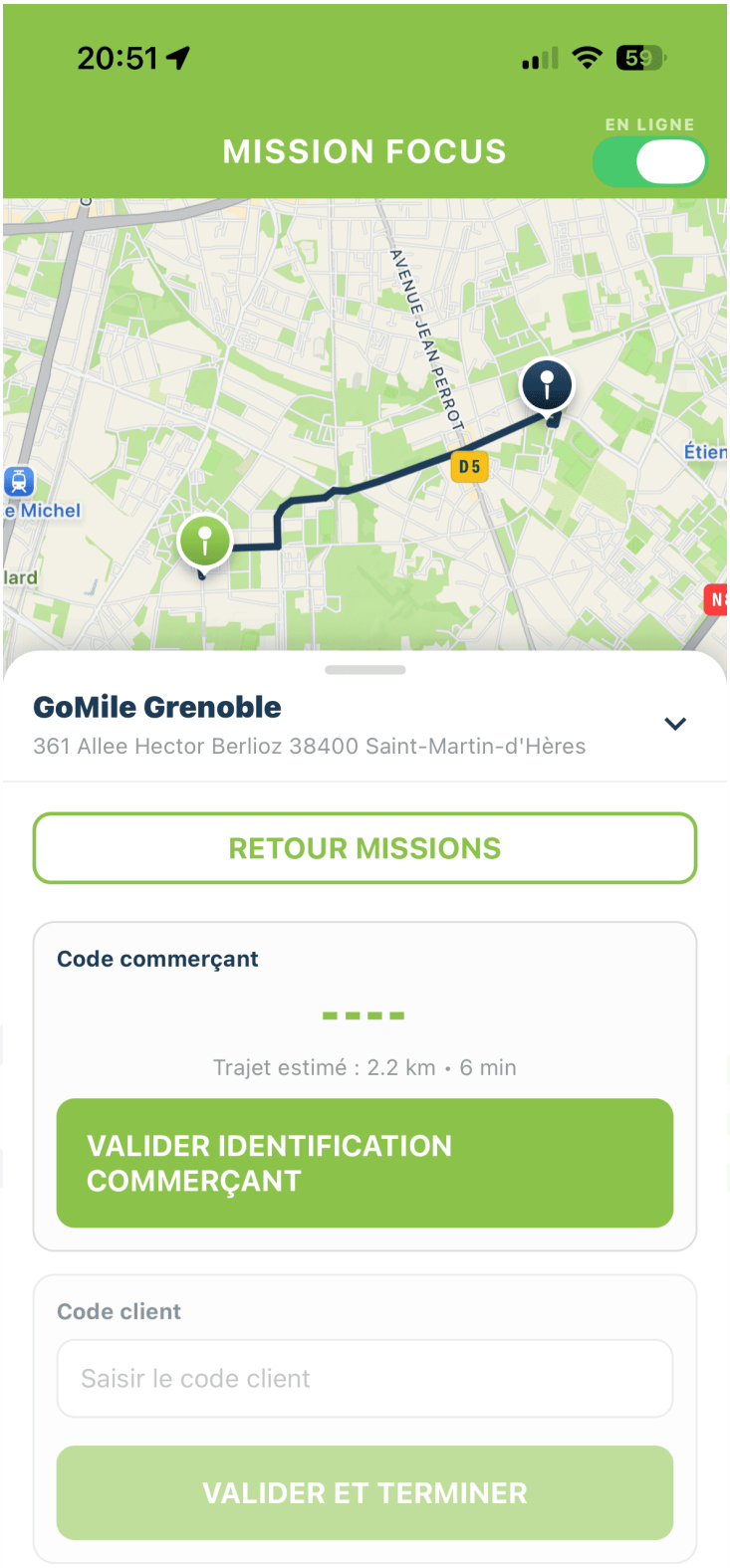


FIGURE 2.5 – Mission Focus — itinéraire et actions métier

1.7 Détail d'une mission

La fiche détail expose les informations métier : adresses, type (FOOD, OTHER...), prix proposé, distance et temps estimé. Les boutons d'acceptation/refus sont clairement identifiés.

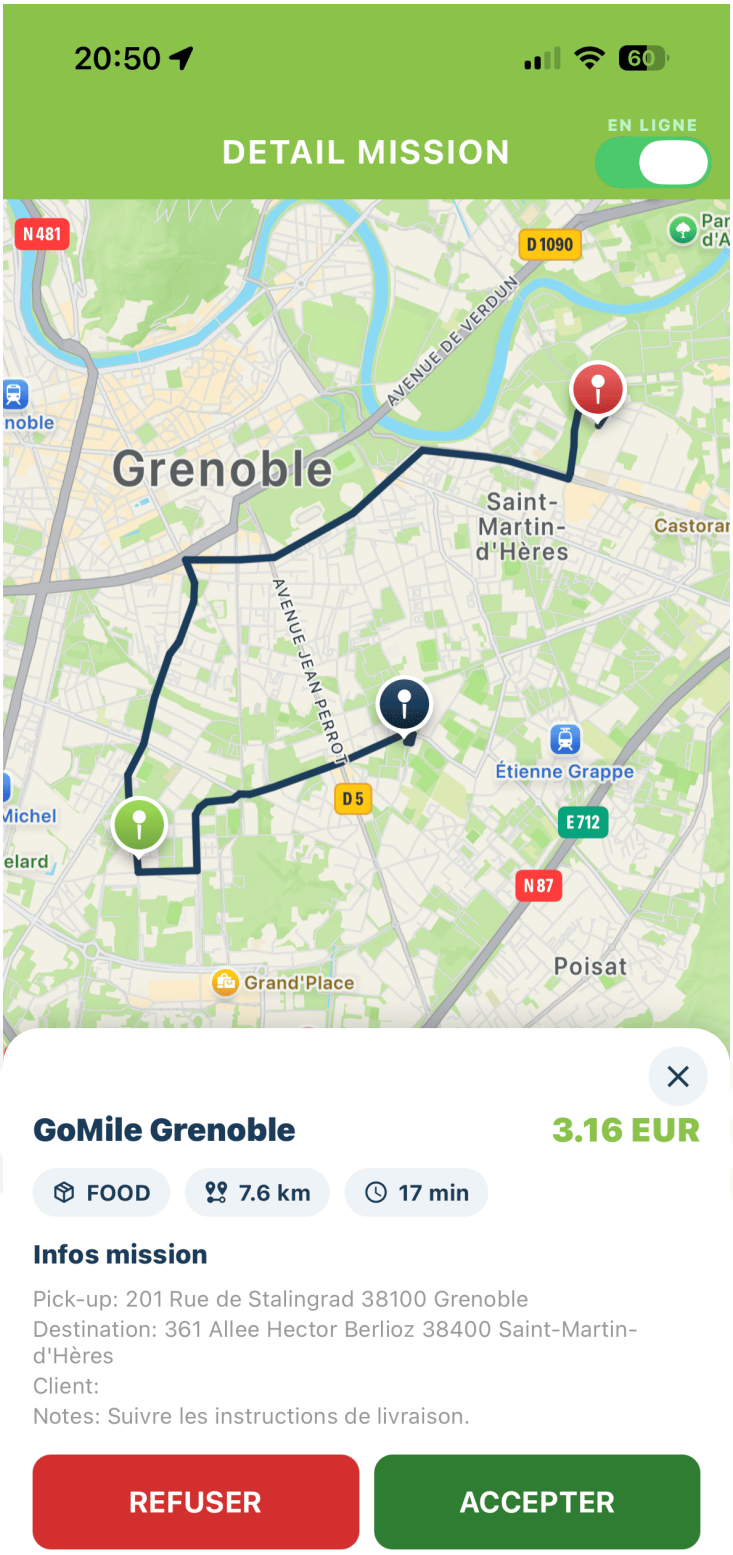


FIGURE 2.6 – Détail mission — infos et actions

1.8 Portefeuille (Wallet)

L'onglet "Gains" montre le solde disponible et les montants en attente. L'utilisateur peut demander un virement depuis le modal de retrait.

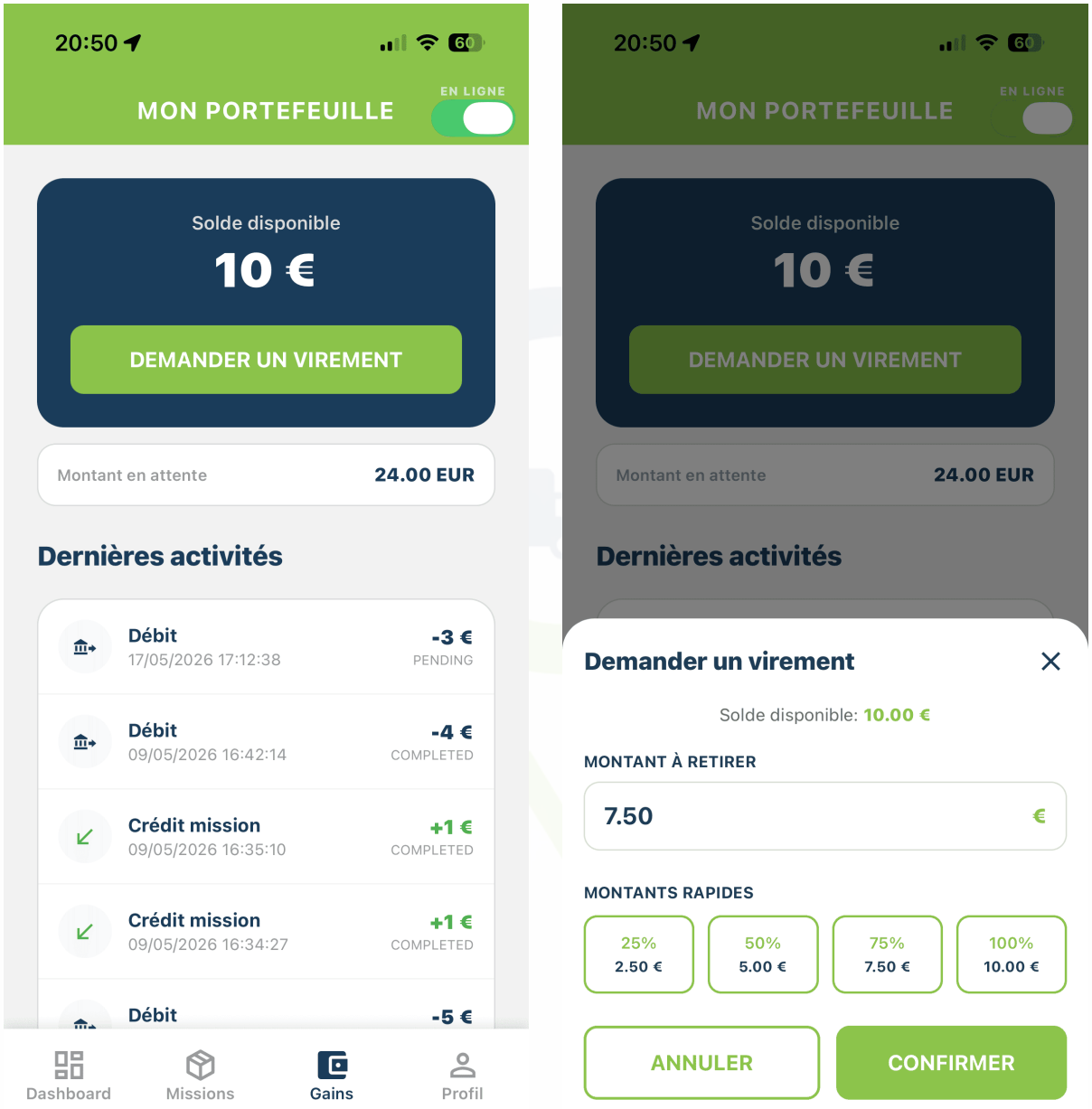


FIGURE 2.7 – Portefeuille — solde et modal de demande de virement

1.9 Profil et vérification KYC

L'écran Profil permet de consulter et modifier les informations du livreur, choisir le véhicule de la session et suivre l'état KYC (non soumis / en attente / approuvé / rejeté). La vignette KYC s'affiche en tête de section.

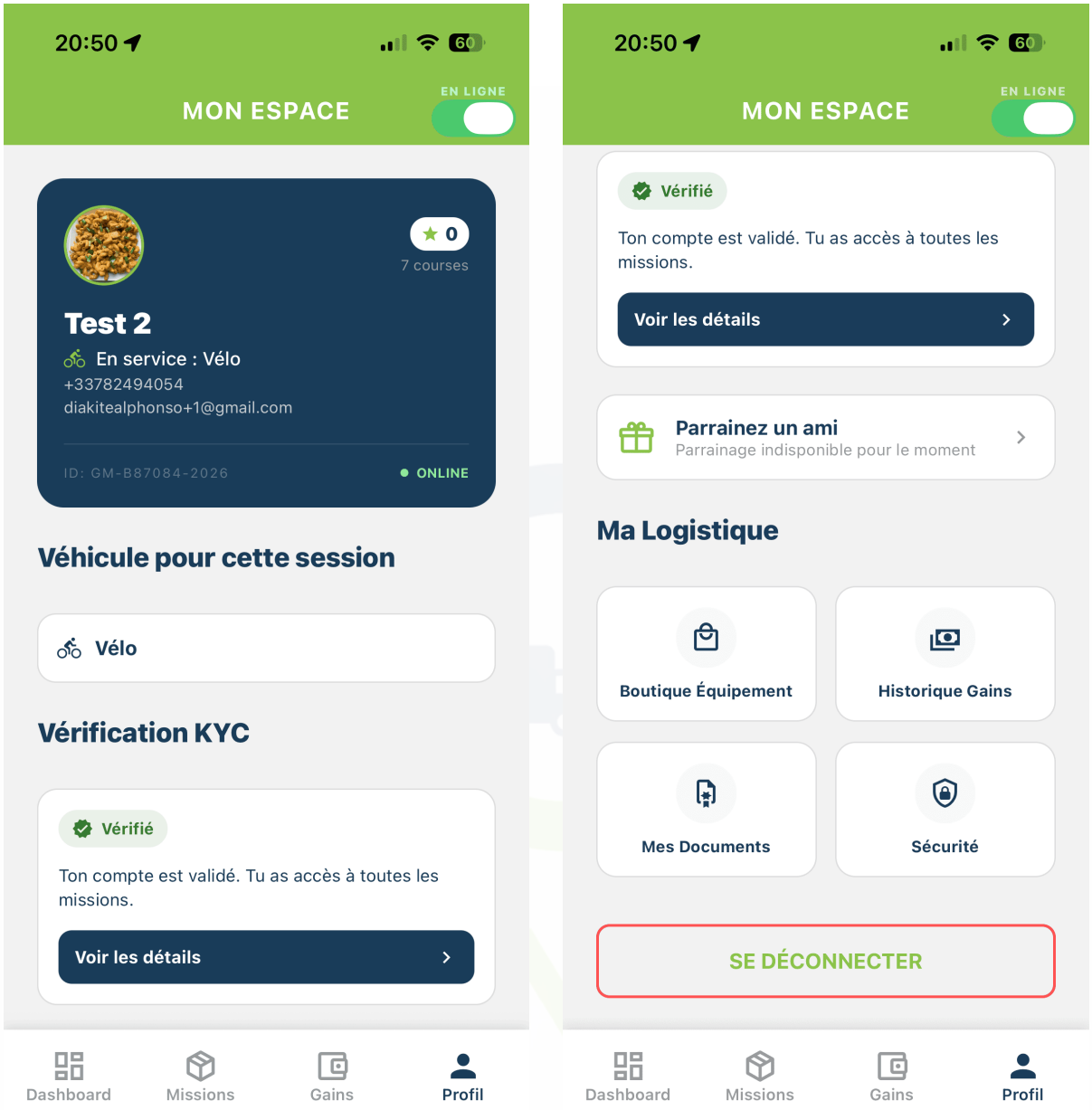


FIGURE 2.8 – Profil — résumé et section KYC / logistique

Chapitre 3

Documentation de conception

1 Backend et API

2 Interface Web

2.1 Partie Administrateur

2.2 Partie Commerciale :

GoMile

3 Application Mobile Livreur

3.1 Rôle de la partie mobile

L'application mobile constitue l'interface dédiée aux livreurs du projet GoMile. Elle leur permet de :

— créer un compte et finaliser l'activation,

— consulter et accepter des missions,

— suivre une livraison avec vérification par code,

— gérer leur profil, leur disponibilité et leurs documents (aka Know Your Customers).

Cette partie a été développée en React Native avec Expo et Expo Go pour accélérer les itérations et faciliter les démonstrations sur téléphone pendant le projet. Les builds prennant beaucoup de temps, nous les consacrons pour les démos de la soutenance.

3.2 Architecture générale et technologies

L'application mobile repose sur les briques suivantes :

React Navigation gestion des écrans d'authentification et des onglets principaux.

Expo accès aux fonctionnalités natives (média, géolocalisation, notifications) avec un cycle de développement rapide.

API backend GoMile authentification, missions, profil, vérification et opérations métier.

Modules externes `@react-native-community/datetimepicker` pour la date, composants média pour photo/document, services d'adressage pour fiabiliser les adresses.

Local storage avec Zustand

Le flux d'inscription traverse plusieurs écrans successifs. Les données doivent donc être partagées entre écrans, conservées pendant la navigation, et réutilisées au moment de l'appel API final. Pour cette raison, nous avons retenu `Zustand` (`useRegistrationStore`) plutôt qu'un simple objet JavaScript local à un écran. Ce choix apporte :

— un état centralisé lisible,

— des mises à jour champ par champ,

— une meilleure robustesse si l'utilisateur revient à l'étape précédente.

La description détaillée des champs arrive ensuite, pour montrer concrètement ce qui est stocké à chaque étape.

3.3 Parcours d'inscription

Point d'entrée et logique de découpage

Depuis l'écran `LoginScreen`, le bouton `Devenir livreur` ouvre le circuit d'inscription.

Le parcours a été progressivement retravaillé lors des suivis avec M. Viardot pour améliorer l'UX : simplification des écrans, réduction des frictions, repositionnement des informations administratives, et validation plus claire des champs. Le flux actuel est volontairement découpé en **trois étapes d'inscription** :

— étape 1 : identité de base,
— étape 2 : informations personnelles complémentaires,
— étape 3 : logistique de livraison.

Un ancien écran de finalisation (ancien `RegisterStep4`) existe encore dans le code, mais il ne fait plus partie du flux UX principal, car les retours d'usage ont montré qu'il vaut mieux finaliser certains éléments après création du compte.

Étape 1 : identité de base (`RegisterStep1.js`)

Champs demandés et types :

— photo de profil : image locale (URI),
— prénom : chaîne de caractères,
— nom : chaîne de caractères,
— email : chaîne de caractères au format email,
— mot de passe : chaîne de caractères,
— confirmation mot de passe : chaîne de caractères.

Modules et UX :

— Le choix de la photo passe par le module `media-picker`, qui encapsule les permissions système et la sélection de média.
— Le champ email utilise un clavier adapté à la saisie email (présence du caractère `"@"` facilitant la saisie).

Validation et stockage Zustand :

— Vérification des champs obligatoires.
— Vérification de la cohérence mot de passe / confirmation.
— Stockage des champs avec le hook personnalisé `useRegistrationStore` : `firstName`, `lastName`, `email`, `password`, `confirmPassword`, `avatarUrl` sont les nom de contenant Zustand que nous avons utilisé.

Étape 2 : identité complémentaire (`RegisterStep2Identity.js`)

Champs demandés et types :

— numéro de téléphone : chaîne normalisable en format FR/international,

— date de naissance : date,

— genre : valeur énumérée (Homme, Femme, Autre).

Modules et UX :

— Le numéro de téléphone utilise un clavier numérique.

— La date utilise `@react-native-community/datetimepicker`, retenu pour sa compatibilité native iOS/Android et son comportement rétractable.

— Le genre est sélectionné avec des boutons à choix explicite.

Validation et stockage Zustand :

— Champs obligatoires contrôlés avant passage à l'étape suivante.

— Stockage dans `useRegistrationStore` avec `phone`, `birthDate`, `gender` utilisés comme contenants.

Étape 3 : logistique de livraison (`RegisterStep2.js`)

Champs demandés et types :

— adresse de résidence : chaîne structurée,

— ville de livraison : chaîne,

— rayon de livraison : entier (km),

— mode de transport : enum (vélo, moto, voiture, utilitaire).

Modules et UX :

— Adresse gérée via un composant d'autocomplétion connecté au backend et à l'API d'adressage gouvernementale "`https://api-adresse.data.gouv.fr/`" pour récupérer des adresses fiables.

— Le rayon impose une saisie numérique uniquement.

— Le mode de transport est proposé sous forme de cartes de sélection pour une lecture rapide.

Normalisation et stockage Zustand :

— Normalisation du téléphone au format international attendu.

— Conversion du rayon en entier exploitable côté API.

— Mapping du transport vers les codes métier (BIKE, SCOOTER, CAR, TRUCK).

— Stockage dans `useRegistrationStore` : `address`, `city`, `zipCode`, `street`, `deliveryCity`, `deliveryRadius`, `transportType`.

Séquence technique d'inscription

Le diagramme suivant décrit le chemin complet : saisie des étapes, création du compte, envoi de l'email de vérification, attente active côté mobile, puis redirection automatique après confirmation.

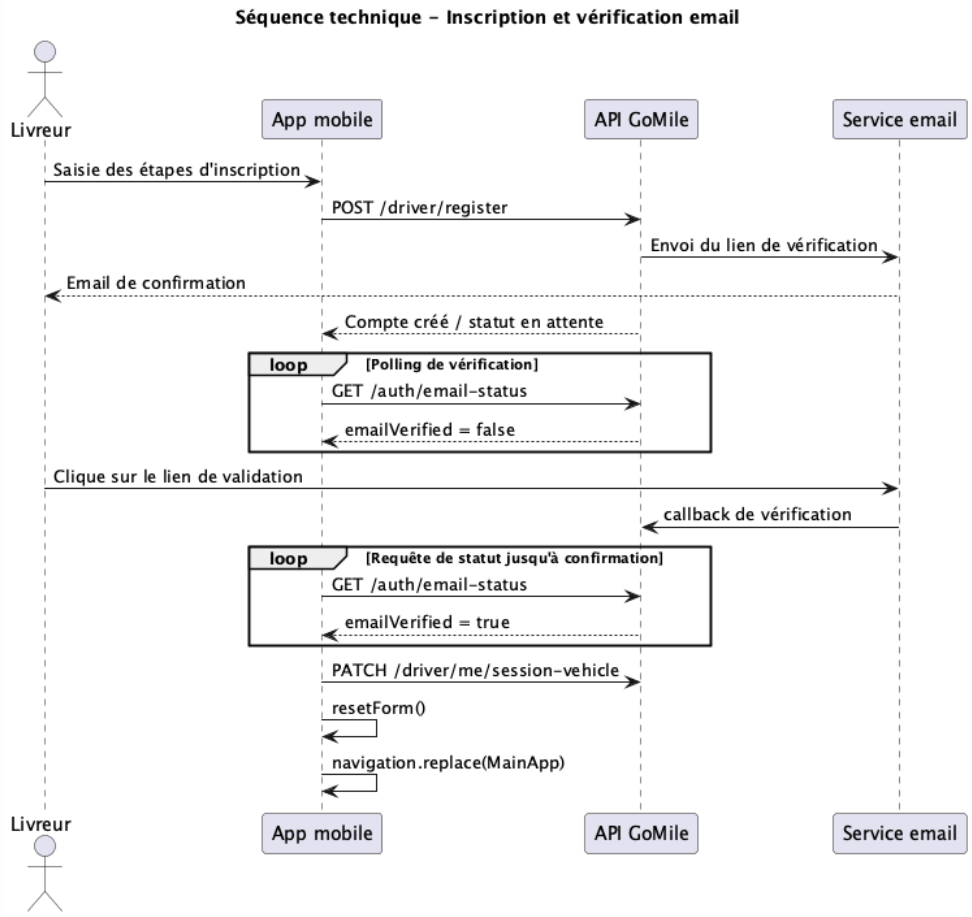


FIGURE 3.1 – Séquence technique d'inscription avec vérification email

3.4 Connexion et vérification email

Après lancement de l'application, l'utilisateur arrive sur `LoginScreen`. Il saisit son identifiant (email) et son mot de passe.

— Si le compte est valide, l'application ouvre directement l'espace principal.

— Si l'email n'est pas vérifié, l'utilisateur est redirigé vers `EmailVerificationScreen`.

Cet écran de vérification interroge périodiquement le backend (chaque 2000ms). En effet, nous attendons une amélioration de la part du backend pour utiliser les websockets pour améliorer les performances. Une fois le mail confirmé via le lien reçu, l'application met à jour la session et redirige automatiquement vers le tableau de bord.

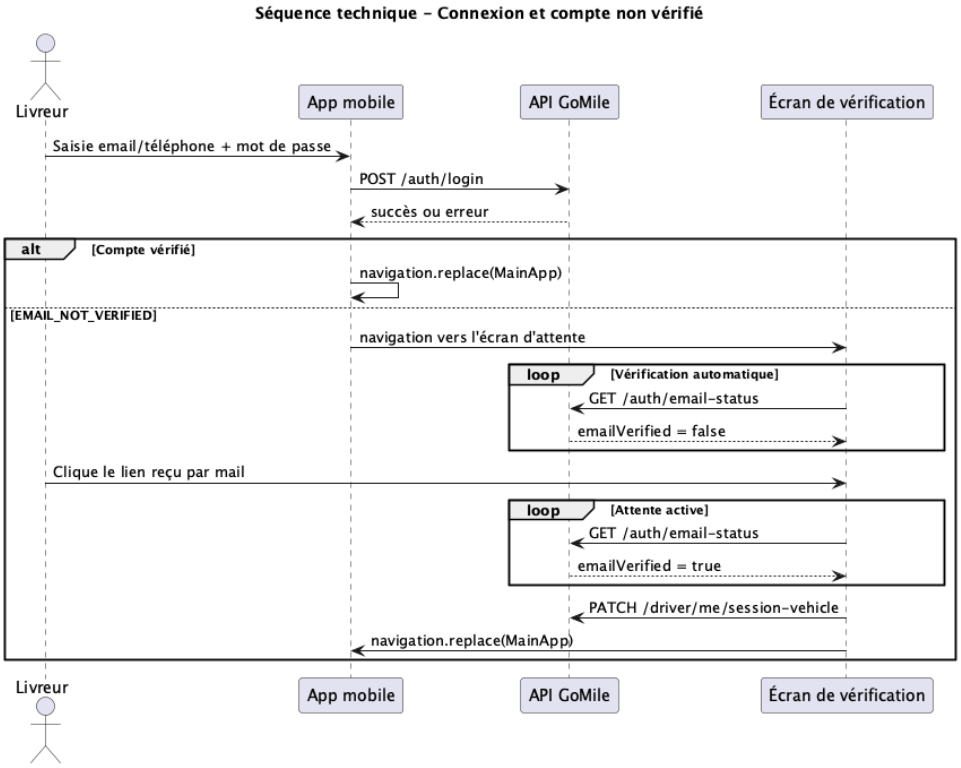


FIGURE 3.2 – Séquence technique de connexion avec gestion de l'email non vérifié

3.5 Réinitialisation du mot de passe

Depuis `LoginScreen`, le bouton `Mot de passe oublié` ouvre le flux de réinitialisation :

— saisie du mail du livreur
— envoi d'une demande de reset,
— réception d'un email,
— définition d'un nouveau mot de passe,
— retour à la connexion.

Cette séparation permet de garder un écran de connexion simple tout en offrant un parcours de récupération complet.

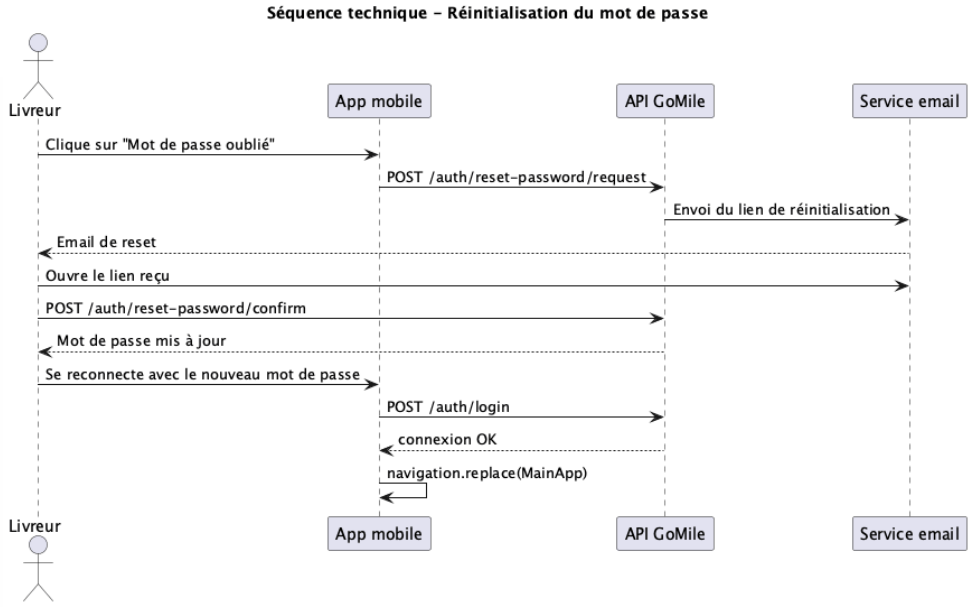


FIGURE 3.3 – Séquence technique de réinitialisation du mot de passe

3.6 Diagramme d'état global authentification

Le diagramme d'état ci-dessous résume les transitions entre les états principaux : connexion, inscription, attente de vérification email, accès au dashboard, et retour après déconnexion ou reset du mot de passe.

3.7 Décisions UX abandonnées ou déplacées

Les évolutions du parcours ont conduit à déplacer plusieurs éléments hors du tunnel d'inscription principal :

- Les documents KYC ne sont plus demandés pendant les trois étapes d'inscription : ils sont traités après création du compte, depuis le profil.
- L'ancien écran de finalisation (ancien `RegisterStep4`) reste présent dans le code, mais n'est plus intégré au flux UX principal.
- La verification du numéro de téléphone (OTP) n'est pas faite pour le moment pour des questions de budget car nous utilisons Twilio qui peut vite être couteux au vu de nos tests quotidiens

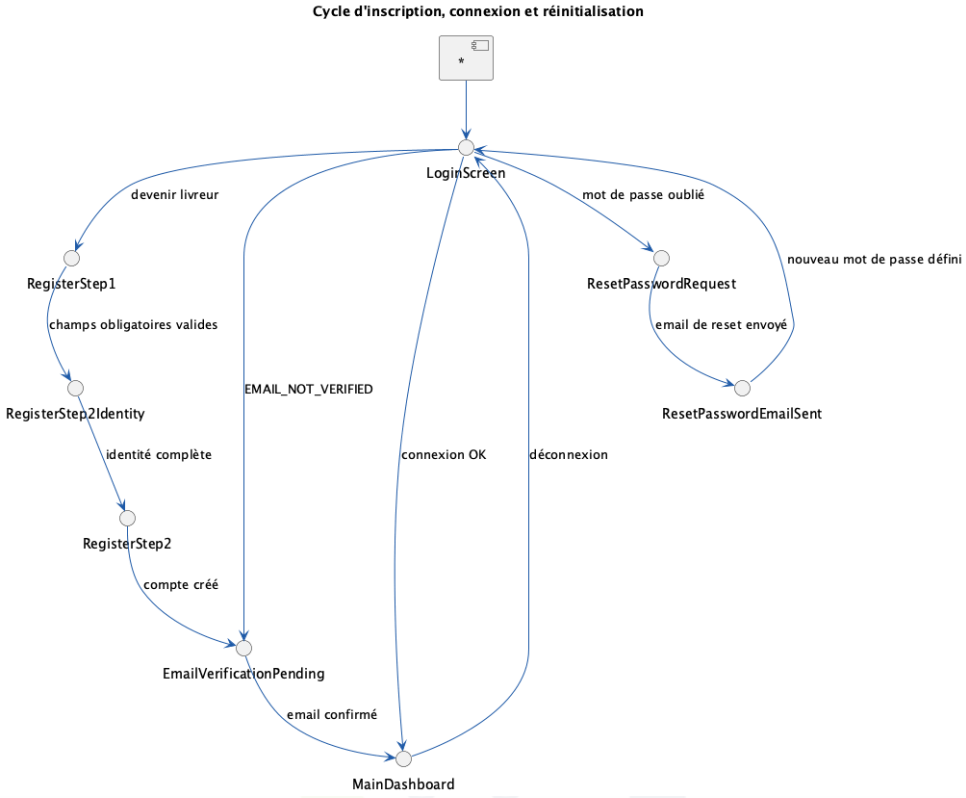


FIGURE 3.4 – Diagramme d'état global du cycle inscription-connexion-reset

3.8 Navigation et écrans post-connexion

Architecture de navigation principale (MainTabs)

Une fois connecté, le livreur arrive sur `MainTabs.js`, qui constitue le cœur de l'application post-authentification. Cette structure repose sur :

React Navigation Bottom Tab Navigator organisation de quatre onglets principaux accessibles en permanence au bas de l'écran.

Stack Navigator implicite chaque onglet peut avoir ses propres écrans enfants (ex : MissionDetailsScreen accessible depuis MissionsScreen).

Push Notifications configuration des notifications à ce niveau, avec enregistrement du token et liaison WebSocket.

Suivi GPS mise à jour de la position du livreur à intervalle régulier pour enrichir les suggestions de missions.

WebSocket (socket.io) connexion établie ici pour recevoir les événements en temps réel (nouvelles missions, changements de statut).

Action rapide globale (très utile, et dont nous sommes beaucoup fier !) un bouton flottant de mission apparaît au-dessus des onglets dès qu'il existe au moins une

mission active ; il renvoie directement vers `MissionFocusScreen` depuis n'importe quel onglet.

Header personnalisé avec toggle en ligne/hors ligne

Le header `Header.js` affiche :

— Le titre de la page courante (ex : «TABLEAU DE BORD», «MISSIONS»).
— Un indicateur `EN LIGNE` / `HORS LIGNE` avec un bouton switch.
— Lorsque le livreur bascule **en ligne**, il devient visible dans les requêtes de mission ; **hors ligne**, aucune mission ne lui est proposée.
— La transition repose sur un appel à `toggleDriverAvailability()` (endpoint PATCH `/driver/me/availability`).
— Sécurité : un livreur ne peut pas se mettre hors ligne s'il a une mission active en cours.

Magasin d'informations partagées : `useAvailabilityStore` (simple toggle Zustand) maintient l'état `isOnline` en mémoire de session.

Écran 1 : Tableau de Bord (Dashboard / TableauDeBord.js)

Purpose et composition

Le Tableau de Bord est l'écran d'accueil principal post-connexion. Il présente :

— **Carte interactive Google Maps** affichant :
  — La position actuelle du livreur (marqueur bleu).
  — Les missions disponibles à proximité (marqueurs rouges).
  — Un cercle de rayon 1 km autour de la position courante.
— **Widget de statistiques rapides** :
  — Gains du jour (format : `XX.XX€`).
  — Nombre de courses effectuées aujourd'hui.
— **Overlay de missions proposées** : défilement horizontal des cartes mission les plus proches, avec bouton «Voir détails».

Logique de localisation et mise à jour

À l'ouverture du Dashboard, on :

1. Demande de permission de géolocalisation (`Location.requestForegroundPermissionsAsync`).
2. Récupère la position courante avec précision `Balanced` sous condition de statut onLine du livreur.
3. Appelle immédiatement `updateDriverLocation(lat, lng)` pour synchroniser la position backend (nécessaire pour la requête géospatiale ST_DWithin).

4. Charge le dashboard via `getDriverDashboard()` pour les statistiques du jour.
5. Lance le hook de polling `useMissionsPoller( fetchAll: false )` qui va rafraîchir les missions disponibles au focus une fois qu'il y a de nouvelle mission avec les websockets.

Intégration des stores et hooks

Stores utilisés :

— `useAvailabilityStore` : lecture de `isOnline` pour afficher/masquer les missions.
— `useMissionStore` : lecture de `availableMissions` pour afficher les marqueurs et les cartes.

Hooks :

— `useMissionsPoller( fetchAll: false )` : fetch unique des missions disponibles au premier focus de l'écran et au passage en ligne.

Endpoints API appelés :

— `getDriverDashboard()` : GET `/driver/me/dashboard` -> retourne `{isOnline, todayEarnings, todayTrips}`.
— `updateDriverLocation(lat, lng)` : PATCH `/driver/me/location` -> envoie la position actuelle.
— `getAvailableMissions()` : GET `/driver/me/missions/available` -> retourne tableau de missions.

Interaction utilisateur

— **Tap sur une carte mission** : ouvre `MissionDetailsScreen` avec les détails complets.
— **Tap sur le bouton GPS** : recentre la carte sur la position courante.
— **Affichage hors ligne** : les missions ne sont pas visibles ; message : «Les missions proposées sont masquées hors ligne».

3.9 Écran enfant : Détails Mission (MissionDetailsScreen)

Purpose et composition

Cet écran affiche les détails complets d'une mission sélectionnée depuis le Tableau de Bord ou depuis l'écran Missions. L'utilisateur peut y accepter ou refuser la mission.

Contenu affiché :

— Carte avec trajet pick-up → destination (Polyline).
— Nom du commerce, adresse pick-up, zone de livraison.

— Rémunération, distance estimée, durée.
— Boutons : REFUSER et ACCEPTER.

Logique d'acceptation et refus

Accepter :

1. Appel API `acceptMission(missionId)` : POST `/driver/me/missions/{id}/accept`.
2. Dispatch action Zustand `acceptMission(missionCard)` qui ajoute la mission à `missionQueue`.
3. Navigation vers `MissionFocusScreen` pour débuter le suivi GPS en direct.

Refuser :

1. Alerte de confirmation : «Cette mission sera retirée de ta liste».
2. Dispatch action Zustand `declineMission(missionId)` qui :
   — Ajoute l'ID à `declinedMissionIds`.
   — Filtre la mission hors de `availableMissions`.
3. Navigation arrière.

Calcul du trajet et géocodage

À l'ouverture :
— Récupère la position courante du livreur.
— Appelle `getDrivingRoute([currentPos, pickupPos, dropoffPos])` pour obtenir la route optimale.
— Affiche distance formatée et durée estimée.

Endpoints API :
— `acceptMissionApi(missionId)` : POST acceptation.
— `getDrivingRoute()` : calcul route (utilise service Google Maps ou routeur local).

3.10 Diagramme de flux : Navigation Tableau de Bord → Détails Mission → Focus

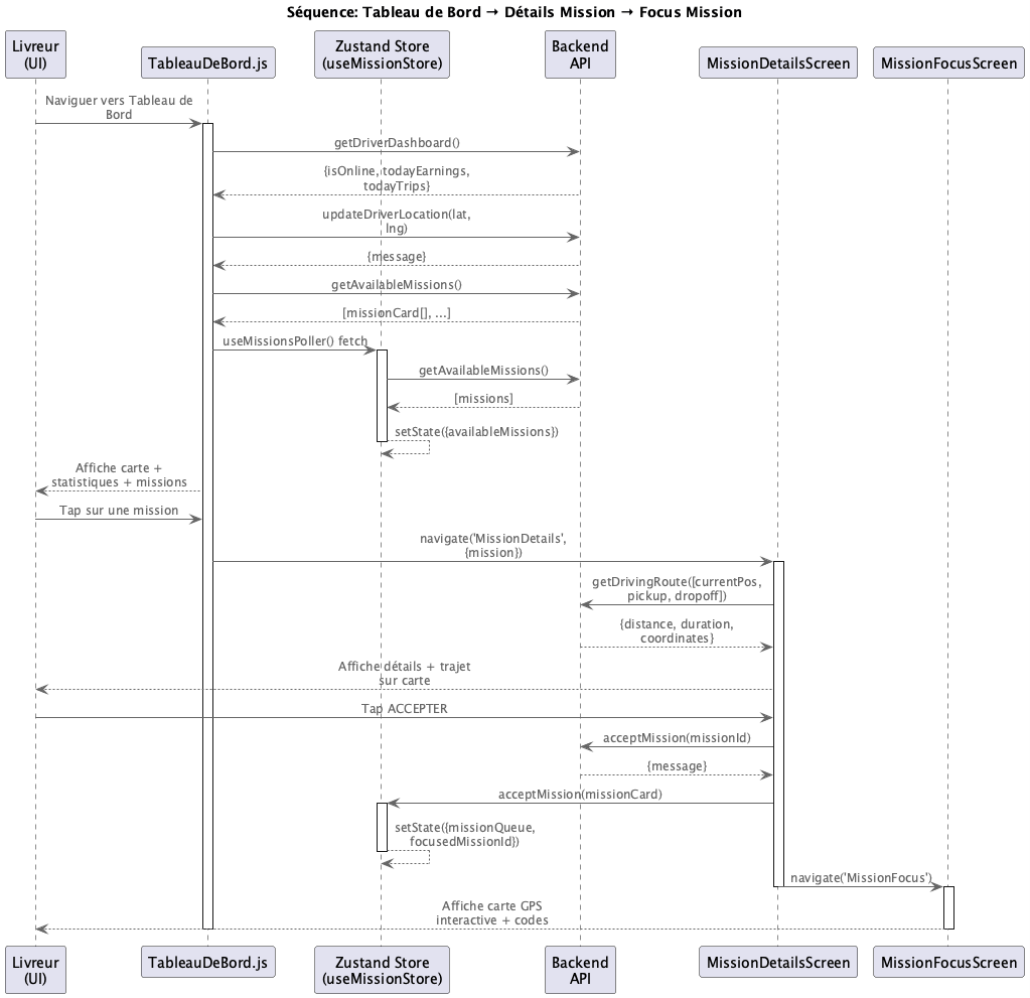


FIGURE 3.5 – Séquence technique : le livreur arrive sur le Tableau de Bord, consulte une mission via MissionDetailsScreen, puis l'accepte pour passer en MissionFocusScreen et débuter le suivi GPS. Les stores Zustand maintiennent l'état entre chaque transition.

3.11 Diagramme d'état : Cycle de vie d'une mission acceptée

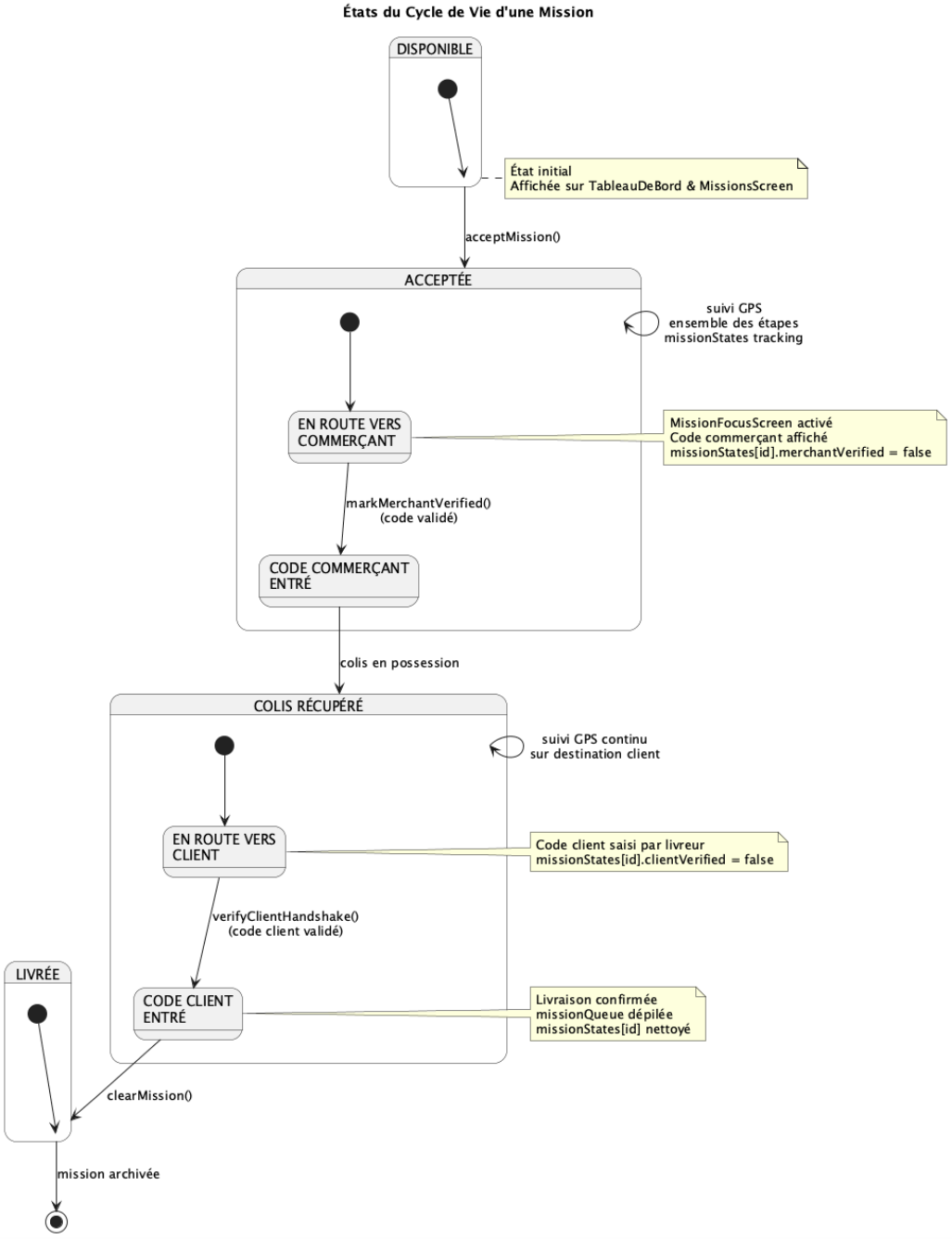


FIGURE 3.6 – États d'une mission : disponible → acceptée (EN ROUTE vers commerçant) → récupérée (RÉCUPÉRÉ colis, EN ROUTE vers client) → livrée (LIVRÉ). Les codes marchant et client actent le passage entre états.

3.12 Écran 2 : Missions (MissionsScreen)

Purpose et organisation par onglets

Cet écran centralise la gestion des missions sous trois onglets logiques :

Notifications Missions résumé du nombre de missions disponibles + horodatage du dernier rafraîchissement.

Onglet 1 : Missions en cours toutes les missions acceptées (état `active` dans `missionQueue`), avec bouton «REPRENDRE LA MISSION» redirigeant vers MissionFocusScreen.

Onglet 2 : Missions à proximité liste complète des missions disponibles avec bouton «VOIR LES DÉTAILS» ouvrant MissionDetailsScreen pour consultation avant acceptation.

Onglet 3 : Historique historique des livraisons (archivées) avec magasins et rémunérations.

Comportement selon le statut en ligne :

— **En ligne** : affichage des trois onglets + banneau informatif.
— **Hors ligne** : banneau unique expliquant que les missions ne sont pas disponibles.

Stores et intégration Zustand

Store `useMissionStore` - lectures :

— `availableMissions` : missions proposées à proximité (GET `/driver/me/missions/available`).
— `activeMissions` : missions en cours acceptées (GET `/driver/me/missions/active`).
— `historyMissions` : missions archivées (GET `/driver/me/missions/history`).
— `lastRefreshAt` : timestamp du dernier fetch.
— `isLoadingMissions` : indicateur de chargement.

Store `useMissionStore` - actions :

— `acceptMission(mission)` : ajoute à `missionQueue` et définit comme focalisée.
— `declineMission(missionId)` : ajoute à `declinedMissionIds`, filtre hors de `availableMissions`.
— `fetchAvailableMissions()` : appel backend pour missions disponibles.
— `fetchAllMissions()` : appel bundle pour tous les états (disponibles + actives + historique).

Polling et refresh

Le hook `useMissionsPoller( fetchAll: true )` gère :

— Fetch unique au premier focus de MissionsScreen.

— Fetch au passage du livreur en ligne.

— Bouton «RAFRAÎCHIR» manuel appelant `refresh()`.

Nouveau système (remplace le polling périodique) :

— Les nouvelles missions arrivent via WebSocket (événement `order:new`) reçu dans MainTabs.

— Le hook ne sert plus au polling périodique, mais au chargement initial et refresh manuel.

— Gain : réduction de la charge réseau, mises à jour en temps réel via événements.

Navigation et actions utilisateur

— **Si on clique sur «VOIR LES DÉTAILS»** : ça ouvre MissionDetailsScreen pour consultation et décision.

— **Clique sur «REPRENDRE LA MISSION»** : on appelle `acceptMission()`, navigue immédiatement vers MissionFocusScreen.

— **Clique «RAFRAÎCHIR»** : on appelle `refresh()` du hook de polling.

3.13 Écran enfant : Focus Mission (MissionFocusScreen)

Purpose : suivi GPS en direct et vérification codes

Cet écran est le cœur de la livraison. Il affiche :

— **Carte interactive** avec :

  — Marqueur bleu : position actuelle du livreur (mise à jour en temps réel).

  — Marqueurs rouges : toutes les missions en cours (pick-up en rouge, livraison en rose).

  — Polyline : trajet calculé vers la cible courante (malheureusement react-native-maps a des performances limitées au vu des tests réalisés et en comparaison de l'appli googleMaps)

— **Panneau bas** (collapsible) contenant :

  — Sélecteur de mission (si plusieurs en cours).

  — Code commerçant (affiché, non saisi).

  — Bouton de validation commerçant.

  — Champ saisie code client (après validation commerçant).

  — Bouton de validation client.

Logique de vérification : deux étapes

Étape 1 : Vérification commerçant

1. Livreur arrive au point de pick-up.
2. Code commerçant est affiché via `getMissionPickupCode(missionId)`.
3. Commerçant valide le code via son interface (pas de saisie livreur ici).
4. Livreur tape le bouton «VALIDER IDENTIFICATION COMMERÇANT».
5. Action `markMerchantVerified(missionId)` active l'étape suivante.
6. État : `missionStates[id].merchantVerified = true`.

Étape 2 : Vérification client

1. Livreur arrive au point de livraison.
2. Champ de saisie code client devient actif (6 chiffres).
3. Client fournit le code (qu'il a reçut via SMS).
4. Livreur saisit le code.
5. Appel API `verifyClientHandshake(missionId, code)` : POST `/driver/me/missions/\{id\}/handshake/client/verify`.
6. Si succès : action `markClientVerified(missionId)`.
7. Alerte : «Mission terminée, Livraison confirmée !».
8. Action `clearMission(missionId)` : dépile de `missionQueue`, met à jour `focusedMissionId`.

Suivi GPS continu

À l'ouverture du MissionFocusScreen :

1. Demande permission géolocalisation.
2. Lance `watchPositionAsync()` : mise à jour position toutes les 5 secondes (distance minimale : 10 m).
3. À chaque nouvelle position :
   — Met à jour `currentPosition` (état local).
   — Recalcule la route vers la cible courante via `getDrivingRoute()`.
   — Affiche distance et durée estimée.
4. Arrêt au démontage du composant.

Sélecteur de missions (si plusieurs en file)

Si `missionQueue.length > 1` :

— Affiche composant `MissionPicker` permettant de basculer entre missions.
— À chaque sélection, `setFocusedMission(id)` met à jour `focusedMissionId`.
— La carte et le panneau se réajustent instantanément ! (magie de useEffect())
— Affichage du statut de chaque mission : EN ROUTE, RÉCUPÉRÉ, LIVRÉ.

Endpoints API utilisés

— `getMissionPickupCode(missionId)` : GET `/driver/me/missions/{id}/handshake/pickup-c`
→ code commerçant.
— `verifyClientHandshake(missionId, code)` : POST `/driver/me/missions/{id}/handshake/client/verify` → validation livraison.
— `getDrivingRoute([coords])` : calcul route.

3.14 Diagramme de flux : Acceptation mission → Focus → Vérification codes

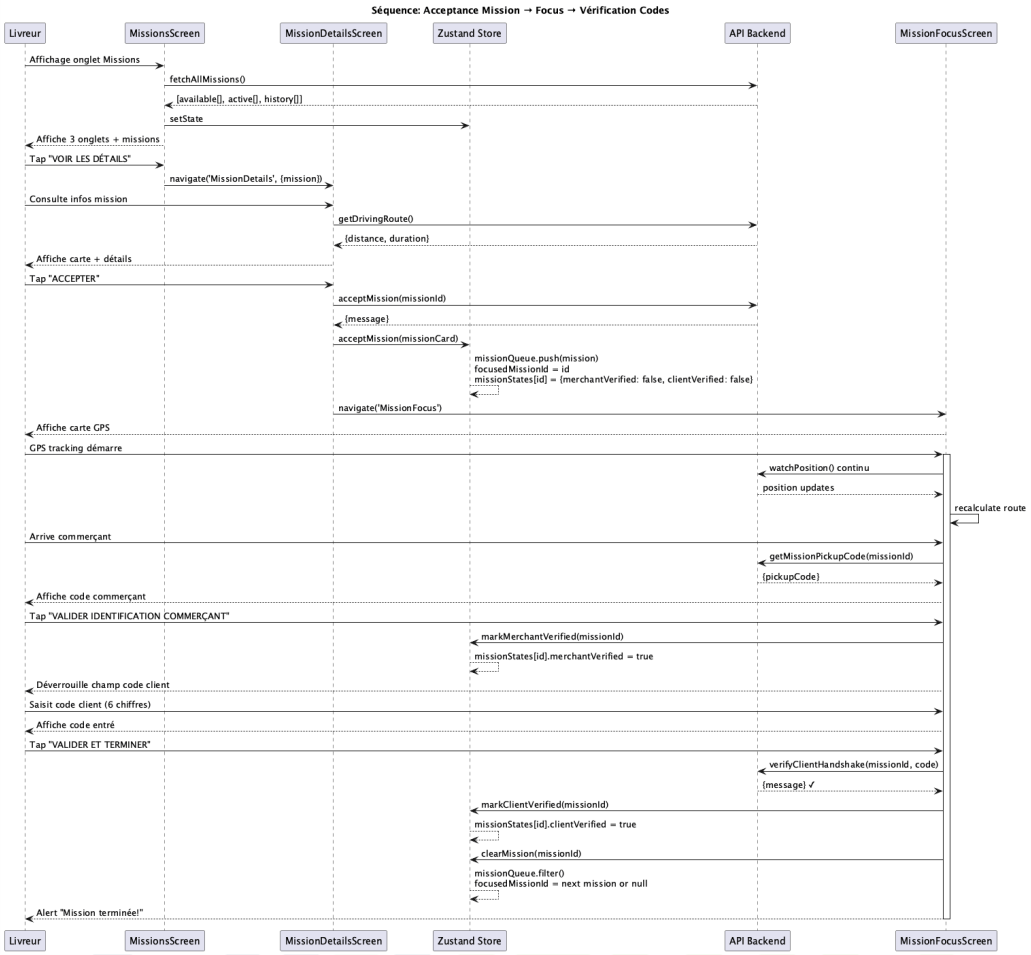


FIGURE 3.7 – Séquence technique : depuis MissionsScreen, le livreur accepte une mission via MissionDetailsScreen. La mission s'ajoute à la file (missionQueue). MissionFocusScreen s'affiche avec suivi GPS. Codes commerçant et client sont vérifiés séquentiellement, puis la mission est archivée.

3.15 Diagramme d'état : Transition états mission dans MissionFocusScreen

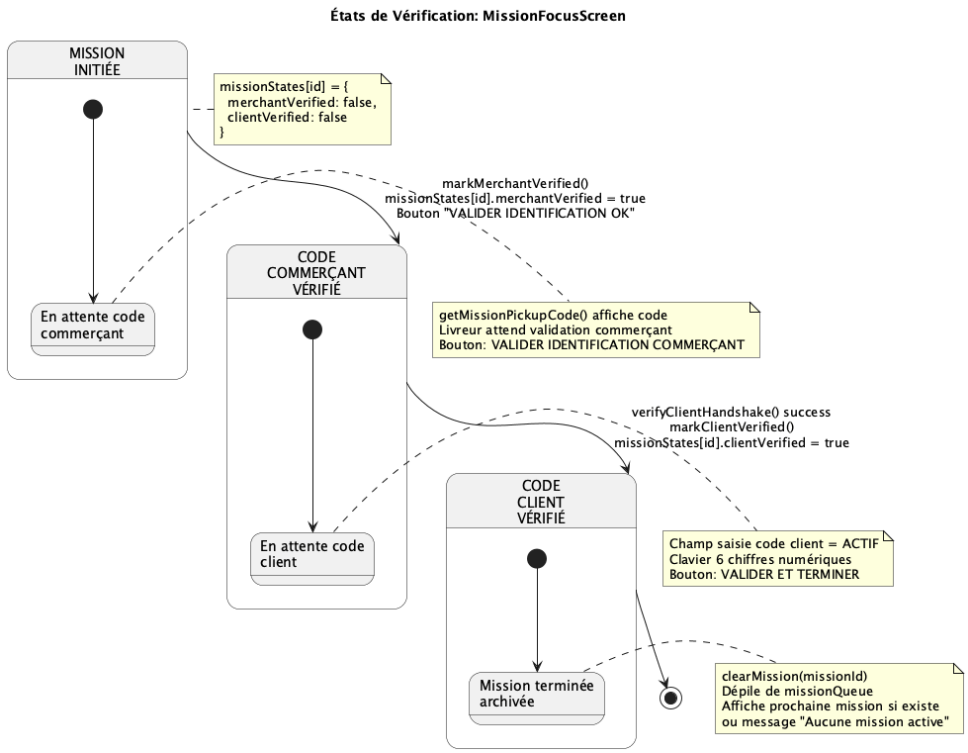


FIGURE 3.8 – États de vérification dans MissionFocusScreen : INITIÉE → COMMERçANT VÉRIFIÉ (merchantVerified=true) → CLIENT VÉRIFIÉ (clientVerified=true) → TERMINÉE. À chaque étape, le panneau se met à jour pour afficher le code suivant ou le message de fin.

3.16 Écran 3 : Portefeuille (WalletScreen)

Purpose : gestion du solde et retrait

Le Portefeuille affiche les gains du livreur et permet les demandes de virement. Composants :

- **Carte solde principal** : affiche le solde disponible et le montant en attente de paiement.
- **Historique des transactions** : liste des crédits et débits avec dates, montants, statuts.
- **Bouton «Demander un virement»** : ouvre modal de retrait.
- **Modal de retrait** : saisie montant ou boutons rapides

Modal de retrait

À l'ouverture du modal :

1. Affiche solde disponible.
2. Champ saisie montant (`TextInput`, clavier décimal).
3. Boutons rapides calculant pourcentages du solde :
   - 25%, 50%, 75%, 100% pré-calculs.
   - Champ se remplit automatiquement.
4. Validation et vérification :
   - Montant doit être > 0.
   - Montant doit être $\leq$ solde disponible.
5. Appel API `requestWithdrawal(amount)` : POST `/driver/me/wallet/withdrawals`.
6. Sur succès : alerte confirmation + rechargement du solde.

Endpoints API

- `getWallet()` : GET `/driver/me/wallet` → `{balance, pendingAmount, currency}`.
- `getWalletEntries()` : GET `/driver/me/wallet/entries` → historique transactions.
- `requestWithdrawal(amount)` : POST `/driver/me/wallet/withdrawals` → demande virement.

3.17 Diagramme de flux : Wallet (solde → retrait → confirmation)

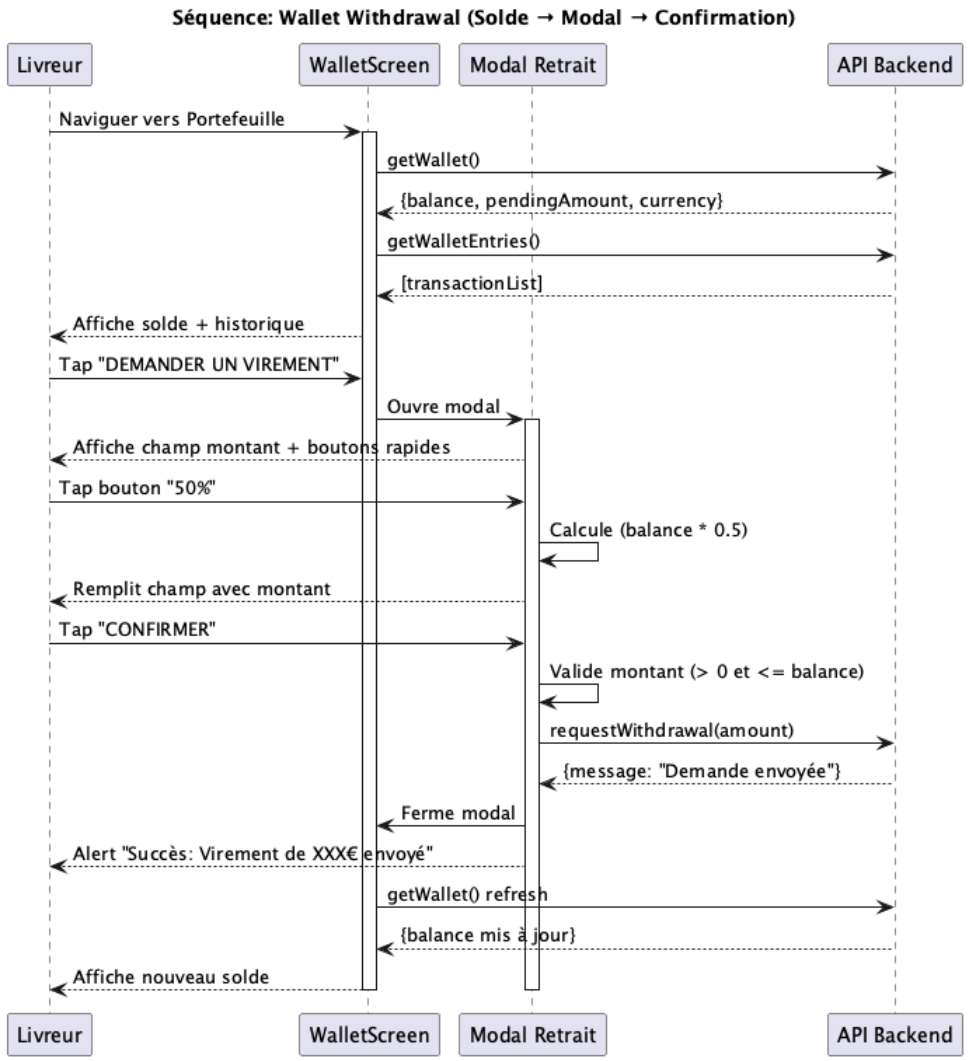


FIGURE 3.9 – Séquence : livreur consulte solde, ouvre modal retrait, calcule montant (rapide ou saisie), valide. API confirme, solde est mis à jour.

3.18 Écran 4 : Profil (ProfileScreen)

Purpose : gestion identité et KYC

Le Profil réunit les informations personnelles du livreur et l'accès au parcours KYC. Composants :

- **Carte identité principale** :
  - Avatar circulaire (stocké sur Amazon AWS S3).
  - Nom et prénom.

  - Note moyenne (rating) et nombre total de courses.
  - Véhicule actuel (BIKE, SCOOTER, etc.).
  - Numéro de code GoMile unique.
  - Statut en ligne (ONLINE / OFFLINE).
- **Sélecteur de véhicule** : change le type de véhicule actif pour la session via `updateSessionVehicle(vehicleType)`.
- **Widget KYC** : affiche le statut de vérification (NOT_SUBMITTED, PENDING, APPROVED, REJECTED) avec badge coloré et CTA vers KYCScreen.
- **Section parrainage** : affiche code de parrainage et bouton pour partager avec amis.
- **Bouton déconnexion** : termine la session, clear tokens, retour écran LoginScreen.

Statut KYC sur ProfileScreen

Chaque état affiche une configuration visuelle distincte :

- **NOT_SUBMITTED** : badge orange, icône alerte, message «Ajoute tes documents», CTA «Compléter mon KYC».
- **PENDING** : badge bleu, icône horloge, message «En cours de vérification», CTA «Voir mes documents».
- **APPROVED** : badge vert, icône checkmark, message «Compte vérifié», CTA «Voir les détails».
- **REJECTED** : badge rouge, icône X, message «Documents refusés», CTA «Corriger mes documents».

Tap sur le widget : redirection vers KYCScreen avec le statut courant.

Endpoints API

- `getDriverProfile()` : GET `/driver/me/profile` → infos perso.
- `updateSessionVehicle(vehicleType)` : PATCH `/driver/me/session-vehicle`.
- `getMyKycStatus()` : GET `/driver/me/kyc` → statut KYC.
- `getMyReferral()` : GET `/driver/me/referral` → code parrainage.
- `logout()` : clear tokens (local storage).

3.19 Diagramme d'état : Profil et transitions KYC

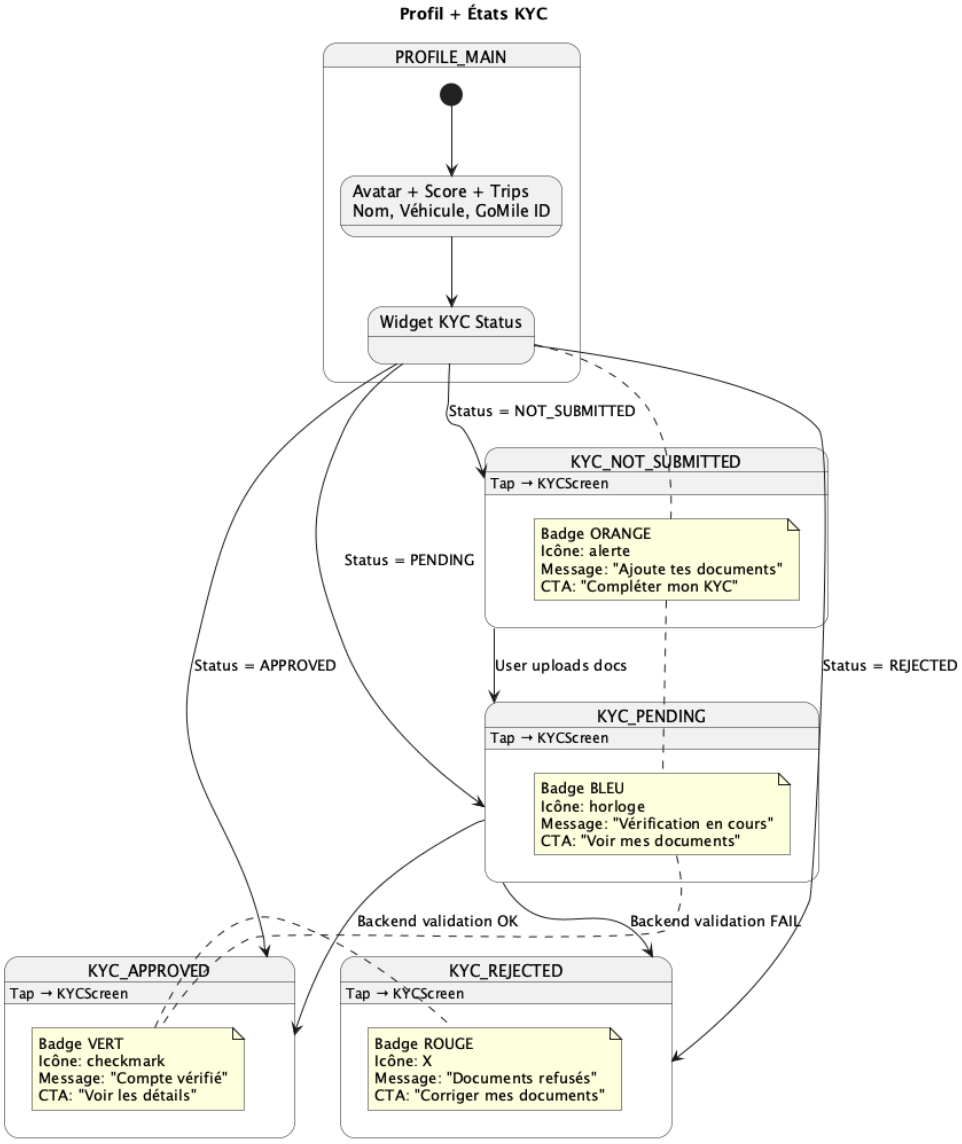


FIGURE 3.10 – État du profil : affichage des infos de base. Widget KYC avec 4 états colorés (NOT_SUBMITTED → PENDING → APPROVED ou REJECTED). Tap sur widget navigue vers KYCScreen.

3.20 Écran enfant : KYC (KYCScreen)

KYCScreen est l'écran dédié à la vérification d'identité et à la gestion des documents. C'est ici que le livreur téléverse les pièces justificatives requises.

Slots et types de documents

GoMile demande 5 emplacements (slots) de documents (Cette partie est succeptible de beaucoup changer car nous avons pas fini de discuter sur la notion de KYC avec les collègues) :

IDENTITY accepte «**Carte nationale d'identité**» (CNI) OU «**Passeport**». Un seul des deux suffit.

DRIVING_LICENSE permis de conduire.

REGISTRATION_CARD carte grise (immatriculation véhicule).

RIB relevé bancaire (IBAN/RIB pour virements).

JUSTIFICATIF_DOMICILE justificatif de domicile de moins de 3 mois.

L'utilisateur peut téléverser des fichiers images (PNG, JPEG) ou PDF.

États de vérification d'un document

Chaque document dans un slot a quatre états possibles :

Non fourni aucun fichier uploadé ; puce grise ; CTA «Ajouter document».

En attente d'examen document uploadé mais pas encore vérifiée par backend ; icône horloge bleu ; puce bleu.

Vérifié backend a validé le document ; icône checkmark vert ; puce vert ; badge «Vérifié ✓».

Refusé backend a rejeté le document ; icône X rouge ; puce rouge ; badge «Refusé — à corriger» + raison de rejet.

Statut KYC global

Le statut global du compte KYC a aussi quatre états :

NOT_SUBMITTED aucun document n'a été soumis ; affichage : badge «KYC à compléter», sous-titre «Téléverse tes documents», icône alerte.

PENDING au moins un document soumis, en attente de vérification backend ; badge «Vérification en cours», sous-titre «Examen sous 48h», icône horloge.

ACCEPTED tous les documents requis ont été validés ; badge «Compte vérifié», sous-titre «Tu as accès à tous les missions», icône checkmark vert.

REJECTED un ou plusieurs documents refusés ; badge «Documents refusés», sous-titre «Corrige-les et resoumets», icône X rouge.

Workflow d'upload de document

Étape 1 : Sélection du fichier

1. Livreur tap sur slot vide ou sur document existant pour modifier.
2. Modal : choix «Caméra», «Galerie photo», ou «Fichier».
3. Livreur sélectionne source et choisit/prend le fichier.
4. Module `media-picker` : applique permissions, retourne URI du fichier.

Étape 2 : Presignage URL S3

1. Appel API `presignDocument(filename, contentType)` : POST `/driver/me/documents/presign`.
2. Backend retourne : `{uploadUrl, fileUrl}`.
3. `uploadUrl` est une URL S3 signée (temporaire, 5 min).
4. `fileUrl` est l'URL finale du fichier stocké.

Étape 3 : Upload S3 direct

1. Utilise `uploadFileToPresignedUrl(uploadUrl, file, contentType)`.
2. Envoie fichier directement vers S3 via l'URL signée (bypass backend, plus rapide).
3. Indicateur loading : «Envoi en cours...».
4. Sur succès : puce de l'état passe à «En attente d'examen» (bleu).

Étape 4 : Liaison document au slot

1. Appel API `createMyDocument(type, fileUrl)` : POST `/driver/me/documents`.
2. Backend enregistre le document lié au slot.
3. État du document : `{id, type, url, verified: false, rejectionReason: null, createdAt}`.

Étape 5 : Soumission KYC (optionnel)

1. Livreur voit tous les documents uploadés.
2. Bouton «SOUMETTRE MA DEMANDE KYC» : appel API `submitMyKyc()` : POST `/driver/me/kyc`.
3. Statut passe à PENDING ; backend commence vérification.

Suppression document

Livreur peut supprimer un document via bouton `deleteDocument(documentId)` : DELETE `/driver/me/documents/\{id\}`.

La puce revient à «Non fourni».

Endpoints API pour KYC

— `getMyKycStatus()` : GET `/driver/me/kyc` → `{status: 'NOT_SUBMITTED'|'PENDING'|'ACCEPTED'|'REJECTED', latestSubmission}`.
— `getMyDocuments()` : GET `/driver/me/documents` → liste des documents uploadés.
— `presignDocument(filename, contentType)` : POST `/driver/me/documents/presign` → `{uploadUrl, fileUrl}`.
— `uploadFileToPresignedUrl(uploadUrl, file, contentType)` : PUT vers S3 (hors API GoMile).
— `createMyDocument(type, fileUrl)` : POST `/driver/me/documents` → enregistre document.
— `deleteDocument(documentId)` : DELETE `/driver/me/documents/\{id\}`.
— `submitMyKyc()` : POST `/driver/me/kyc` → soumet au backend pour vérification.

Diagramme de flux : Upload document KYC complet

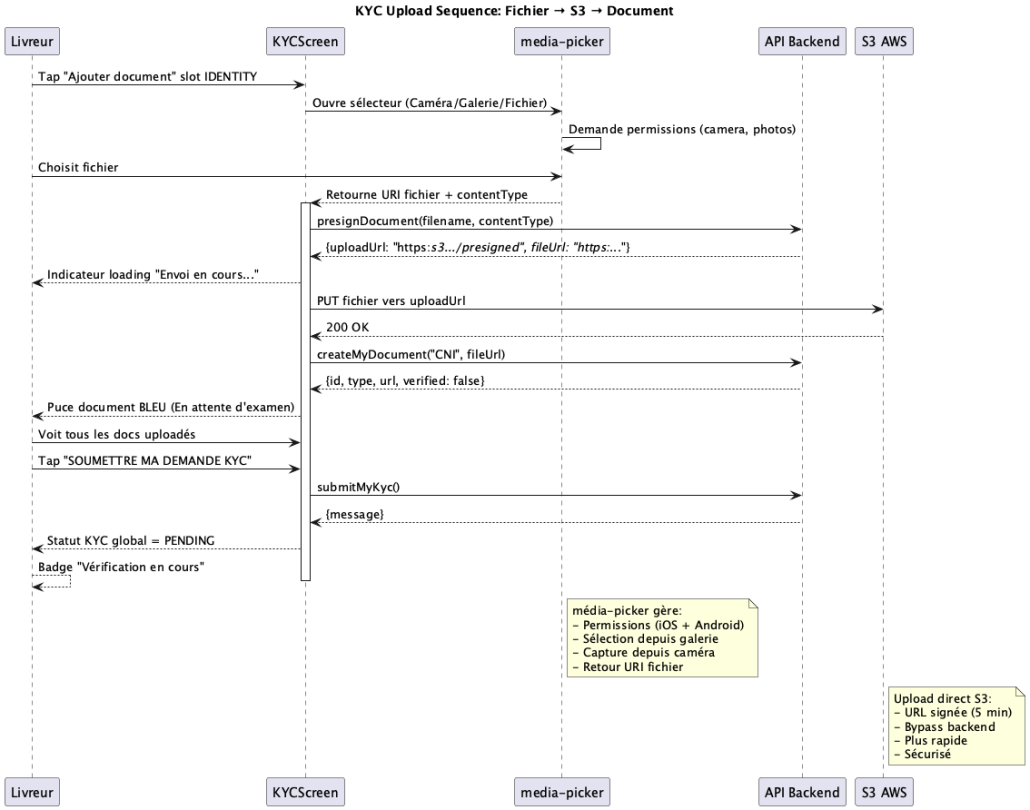


FIGURE 3.11 – Séquence technique : livreur sélectionne fichier → presignage URL S3 → upload direct S3 → création document backend. État passe à «En attente d'examen». Soumission optionnelle vérrouille les modifications.

Diagramme d'état : KYC global + transitions documents

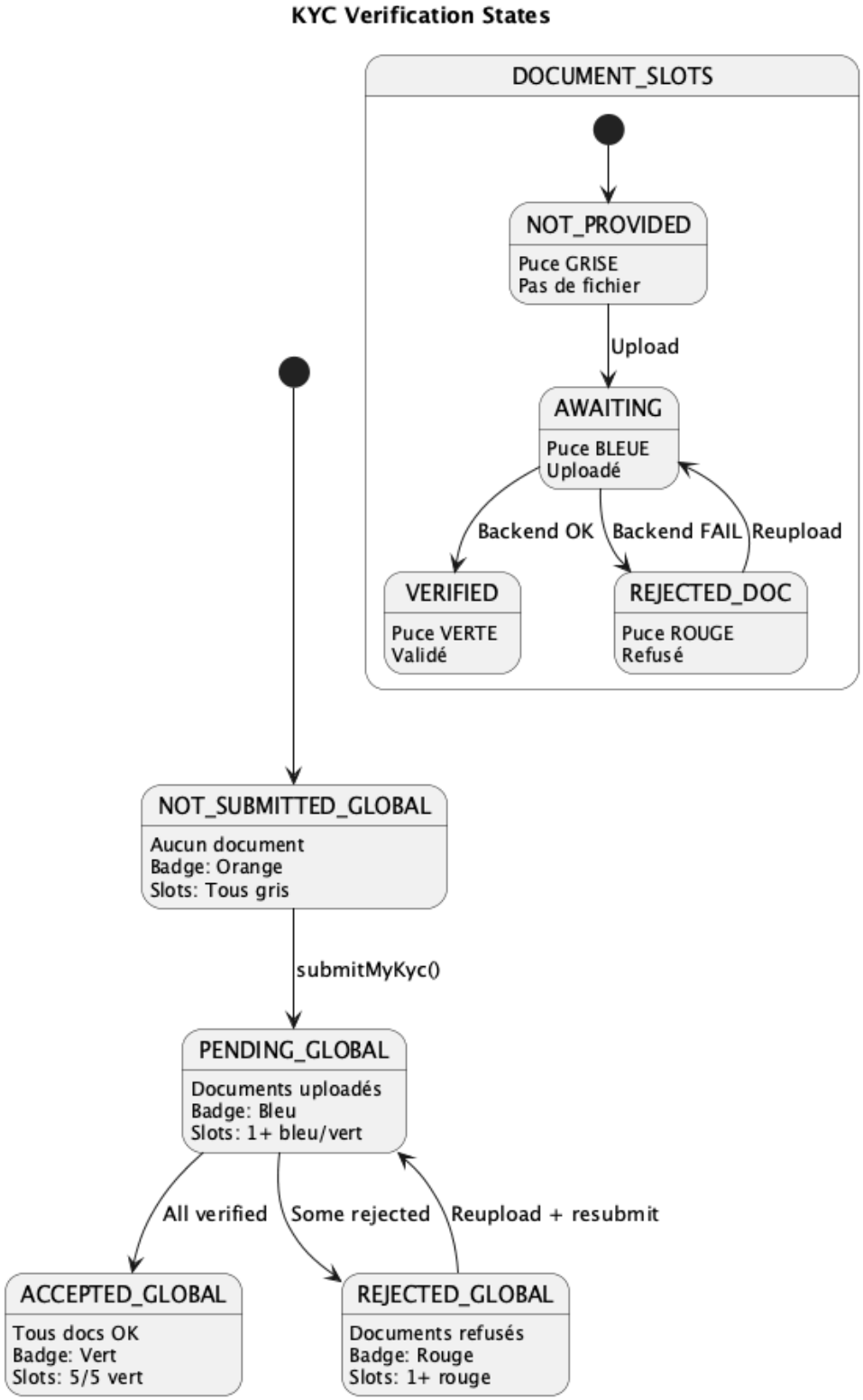


FIGURE 3.12 – États : NOT_SUBMITTED → PENDING (documents uploadés) → ACCEPTED (tous vérifiés) ou REJECTED (révision nécessaire). Chaque slot a ses propres états (Non fourni, En attente, Vérifié, Refusé). Livreur peut reupload et resubmit.

3.21 Services et stores partagés

Choix d'architecture pour la persistance des missions

Au début du projet, l'état des stores Zustand était partiellement sauvegardé localement dans un fichier JSON sur le téléphone. L'idée était simple : si l'application était relancée, on relisait cette sauvegarde locale pour reconstruire l'état de navigation et de mission. Cette approche pouvait sembler pratique pour un prototype, mais elle introduisait un risque important : si le téléphone du livreur s'éteignait, redémarrait, ou perdait son stockage temporaire pendant une livraison, l'application pouvait restaurer une mission qui n'existait plus réellement côté serveur. On se retrouvait alors avec une *mission fantôme* : l'interface affichait une livraison en cours alors que le backend ne pouvait plus garantir la cohérence de cet état.

Après concertation avec l'équipe, nous avons donc décidé de déplacer la source de vérité vers le backend. Désormais, les stores Zustand restent utiles pour l'interface et la synchronisation en mémoire de session, mais l'état métier critique est reconstruit à partir des API serveur au démarrage et lors des rafraîchissements. Ce choix rend l'application plus robuste face aux pertes de contexte, supprime les divergences entre téléphone et serveur, et garantit qu'une mission en cours reste traçable même si le mobile se coupe brutalement ou devient indisponible.

En pratique, cela signifie que : 1) le téléphone n'est plus responsable de conserver durablement l'état d'une mission ; 2) le backend garde l'historique, les statuts et les vérifications ; 3) au relancement, l'application interroge le serveur pour reconstruire un état fiable au lieu de faire confiance à une sauvegarde locale possiblement obsolète.

useMissionStore (Zustand)

Le store central pour la gestion des missions :

missionQueue tableau des missions acceptées en cours de livraison.

focusedMissionId ID de la mission actuellement suivie (MissionFocusScreen).

missionStates objet : `{[missionId]: {merchantVerified, clientVerified}}`.

availableMissions missions proposées à proximité.

activeMissions missions acceptées (alias de missionQueue pour l'affichage).

historyMissions missions archivées/livrées.

lastRefreshAt timestamp dernier fetch.

isLoadingMissions indicateur chargement.

Actions principales :

— `acceptMission(mission)` : ajoute à missionQueue, définit focusée.
— `declineMission(missionId)` : ajoute à declinedMissionIds.
— `markMerchantVerified(id)` : marque commerçant vérifié.
— `markClientVerified(id)` : marque client vérifié.
— `clearMission(id)` : dépile mission, met à jour focusedMissionId.
— `setFocusedMission(id)` : bascule mission focalisée.
— `fetchAvailableMissions()` : GET API missions disponibles.
— `fetchAllMissions()` : GET API tous les états (disponibles + actives + historique).
— `initStore()` : restaure missions actives au démarrage.

useAvailabilityStore (Zustand)

Store simple pour le statut en ligne/hors ligne :
— `isOnline` : booléen état courant.
— `setOnlineStatus(boolean)` : action mise à jour.
— `toggleOnlineStatus()` : toggle action.

Intégration : tout écran lit `isOnline` pour afficher/masquer les éléments mission-dépendants.

useMissionsPoller (Hook personnalisé)

Hook gérant le fetch des missions avec filtrage par écran :
— Paramètre `{ fetchAll: boolean }`.
— `fetchAll = false` : dashboard uniquement missions disponibles (rapide).
— `fetchAll = true` : MissionsScreen charge tous les états.
— Fetch unique au premier focus de l'écran OU passage en ligne.
— Bouton refresh manuel via retour action `{ refresh() }`.
— Les nouvelles missions arrivent aussi via WebSocket (events `order:new` dans MainTabs).

socketService (WebSocket)

Connexion socket.io au backend pour événements temps réel :

connectSocket(token) établit connexion ; emmet `join-driver` avec driverId.

Events reçus — `order:new` : nouvelle mission disponible → fetch + notification push locale.
— `driver:status` : changement statut driver côté serveur (ex : forcer offline).

disconnectSocket() ferme connexion (logout ou app termination).

Socket reste actif pendant toute la durée de session (MainTabs et enfants).

driver-client.ts (API endpoints)

Tous les appels API sont centralisés ici. Endpoints post-connexion :

— `getDriverDashboard()` : GET `/driver/me/dashboard`.
— `toggleDriverAvailability()` : PATCH `/driver/me/availability`.
— `updateDriverLocation(lat, lng)` : PATCH `/driver/me/location`.
— `getAvailableMissions()` : GET `/driver/me/missions/available`.
— `acceptMission(id)` : POST `/driver/me/missions/\{id\}/accept`.
— `getActiveMissions()` : GET `/driver/me/missions/active`.
— `getMissionHistory()` : GET `/driver/me/missions/history`.
— `getMissionPickupCode(id)` : GET `/driver/me/missions/\{id\}/handshake/pickup-code`.
— `verifyClientHandshake(id, code)` : POST `/driver/me/missions/\{id\}/handshake/client/verify`.
— `getWallet()` : GET `/driver/me/wallet`.
— `getWalletEntries()` : GET `/driver/me/wallet/entries`.
— `requestWithdrawal(amount)` : POST `/driver/me/wallet/withdrawals`.
— `getDriverProfile()` : GET `/driver/me/profile`.
— `updateSessionVehicle(vehicleType)` : PATCH `/driver/me/session-vehicle`.
— `getMyKycStatus()` : GET `/driver/me/kyc`.
— `getMyDocuments()` : GET `/driver/me/documents`.
— `presignDocument(filename, contentType)` : POST `/driver/me/documents/presign`.
— `createMyDocument(type, fileUrl)` : POST `/driver/me/documents`.
— `deleteDocument(id)` : DELETE `/driver/me/documents/\{id\}`.
— `submitMyKyc()` : POST `/driver/me/kyc`.

Gestion d'erreurs centralisée : redirection si token invalide, parsing des erreurs backend, retry logic.

Récapitulatif : flux complet d'une session livreur

1. Connexion : LoginScreen → EmailVerificationScreen → MainTabs.
2. MainTabs initialise : notifications push, WebSocket, position GPS, fetch missions initiales.
3. Livreur navigue entre onglets (TableauDeBord, Missions, Portefeuille, Profil).
4. Accepte une mission → MissionDetailsScreen → MissionFocusScreen.

5. Suivi GPS en direct, codes vérifiés séquentiellement.
6. Mission livrée → archivée, gains crédités au portefeuille.
7. Livreur consulte portefeuille, peut demander retrait.
8. Livreur manage profil, KYC, données personnelles.
9. Déconnexion : suppression tokens, fermeture WebSocket.

4 Tests de l'application mobile

Le dossier `tests/` de l'application mobile n'a pas été laissé de côté. Il sert à protéger les briques qui font réellement tenir l'app au quotidien : les stores Zustand, les clients API, les composants réutilisables, et les hooks qui pilotent les missions, la connexion ou l'upload de fichiers.

Deux fichiers de setup rendent cette suite fiable : `tests/setup/jest.setup.js` installe les mocks nécessaires pour Expo, la géolocalisation, les notifications, les cartes, la navigation et le TurboModule bridge React Native ; `tests/setup/jest.setup.afterFramework.js` remet à zéro les mocks et `global.fetch` avant chaque test pour éviter les effets de bord entre cas de test. C'est ce qui permet de tester des modules qui, en production, dépendent du natif ou du réseau, sans rendre les tests fragiles.

4.1 Ce qui est testé

Le contenu du dossier `tests/` couvre trois grands blocs.

1. Les composants réutilisables. On y trouve des tests pour `Header`, `BalanceCard`, `TransactionItem`, `GoMileButton`, `GoMileInput`, `MultiOptionGrid`, `OptionCard`, `ProgressBar`, `ScreenWrapper`, `SectionTitle` et `SmartTouch`. Ces tests vérifient des choses très concrètes : affichage du titre, rendu du solde en euros, gestion de l'état loading, sélection multiple, gestion du clavier, fermeture du sélecteur de date, ou encore blocage d'une action quand un bouton est en chargement.

2. Les stores et utilitaires métier. La suite couvre `useAvailabilityStore`, `useRegistrationStore`, `useMissionStore`, `auth-storage`, `api-errors`, `auth-contracts`, `auth-messages`, `geocode` et `socketService`. Là, les tests vérifient que l'état est bien restauré, que les missions ne sont pas dupliquées, que les erreurs API restent stables, que la géocodification met en cache ses résultats, et que la socket réagit correctement à une connexion, à une déconnexion ou à une tentative de reconnexion.

3. Les clients réseau et hooks critiques. Les tests d'intégration valident `auth-client`, `driver-client`, `upload-client` et `useMissionsPoller`. C'est la partie la plus importante pour la robustesse, parce qu'elle vérifie les vrais scénarios métier : connexion, rafraîchissement de session, passage de l'état email non vérifié, récupération des missions, validation commerçant/client, upload presigné vers S3 et déclenchement d'un refresh manuel.

4.2 Ce que montrent les résultats

Le passage de test complet est propre : **24 suites passées sur 24**, pour **276 tests passés sur 276**, sans snapshot. La couverture confirme aussi la logique du projet : les dossiers `src/store` et une partie de `src/services` sont très bien couverts, car ils portent la logique métier la plus sensible, tandis que plusieurs écrans d'interface restent à 0% de couverture directe parce qu'ils sont surtout validés par les composants et les hooks qu'ils consomment.

Autrement dit, le plan de test n'essaie pas de tout sur-tester visuellement. Il protège surtout ce qui casserait vraiment l'expérience livreur : un token mal géré, une mission qui réapparaît mal, un bouton qui ne déclenche rien, un upload qui échoue sans message, ou une mission en cours qui perd son état.

4.3 Lecture rapide du dossier de tests

On peut lire le dossier `tests/` comme une carte de confiance du mobile :

— `tests/setup/` : installe les mocks et stabilise l'environnement Jest.
— `tests/components/` : protège le rendu et les interactions des briques visuelles.
— `tests/unit/` : verrouille les stores, constantes et helpers locaux.
— `tests/integration/` : valide les flux réseau simulés, là où l'app peut casser en production.

Au final, ce dossier de tests documente bien l'intention de l'équipe : garder une app mobile simple à faire évoluer, mais difficile à casser sans que les tests le signalent immédiatement. Nous effectuerons bien évidemment des tests sur les interfaces graphiques et e2e quand le temps nous le permettra, idéalemment après les examens et avant la soutenance du 20/05

4.4 Couverture des tests (coverage)

Les résultats de couverture générés lors du run des tests sont résumés ci-dessous. Les couleurs permettent d'identifier rapidement les zones critiques : vert $\geq 90\%$, orange $[70\%, 90\%[$, rouge $< 70\%$.

Cible	Stmts	Branch	Funcs	Lines
All files	23.19%	14.37%	27.46%	23.56%
lib	72.56%	51.87%	79.51%	73.97%
src/store	93.47%	57.81%	87.80%	97.14%
src/services	67.85%	47.05%	50.00%	69.23%
src/components	39.28%	37.03%	46.34%	38.88%
src/hooks	94.73%	66.66%	100.00%	93.33%
driver-client.ts	93.44%	62.06%	100.00%	93.44%
upload-client.ts	92.45%	75.00%	100.00%	92.00%
socketService.js	67.85%	47.05%	50.00%	69.23%
useMissionStore.js	97.43%	57.81%	96.55%	100.00%
MainTabs.js	0.00%	0.00%	0.00%	0.00%

Légende : bon niveau de couverture (>= 90%), couverture moyenne (70%–90%), attention (< 70%).

GoMile